Telnet Remote-Management Lab: RFC 854 NVT, RFC 1143 Option Negotiation, IOS-Style VTY Server, and Plaintext vs TLS Sniffing

for Computer Networks Laboratory

Name: Subham Bhat

Class: BCSE III

Roll: 002410501020

Group: A1

College: Jadavpur University

Contents

1 Introduction **3**

1.1 What Was Built 3

2 The Protocol **4**

2.1 Local and Remote Log-in 4

2.2 The Network Virtual Terminal and Its Commands 5

2.3 Options 6

3 System Architecture **6**

4 Implementation **7**

4.1 The NVT Codec as a State Machine 7

4.2 Option Negotiation: Why a Naive Implementation Loops 8

4.3 The Server: VTY Lines and an IOS-Style Shell 9

4.4 Security: Sniffing, TLS and Brute-Force Protection 10

5 Advanced Features **12**

5.1 High Availability: A Router Cluster with Session Resume 12

5.2 AAA: Central Login and Accounting with RADIUS 14

5.3 LINEMODE, Nagle's Algorithm and Tinygrams 16

6 Experiments, Verification and Results **17**

6.1 Running the Module 17

6.2 Automated Verification 17

6.3 Observations 18

6.4 Use in the Integrated Main Project 18

7 Conclusion **19**

AI Acknowledgement **19**

1 Introduction

Remote log-in lets a user at one machine work on another as if the keyboard and screen were attached to it directly. The oldest standard for this on the Internet is **TELNET** (**TEL**etype **NET**work), specified in **RFC 854** (1983). It runs over **TCP port 23** and is still used to manage routers and switches on isolated lab networks, which is exactly how router `JU-Main-R` was configured in Assignment 6.

Telnet has three ideas worth studying:

- **The Network Virtual Terminal (NVT):** both ends translate their local character set into one agreed 7-bit ASCII representation with fixed end-of-line rules, so neither side needs to know the other's terminal.
- **In-band commands:** control functions (Interrupt Process, Erase Character, Are You There, ...) and option negotiation travel in the same TCP byte stream as the data, introduced by the escape byte **IAC = 255**.
- **Symmetric option negotiation:** either side may offer or request features such as remote echo or window-size reporting with `WILL`, `WONT`, `DO` and `DONT`. A careless implementation can loop forever; **RFC 1143** (the "Q method") fixes this.

Its weakness is equally instructive: **everything, including the password, crosses the network in clear text**. This is why SSH replaced it.

1.1 What Was Built

This module is a complete Telnet laboratory written from the RFCs in pure Java 11 with no external libraries:

1. **Protocol engine** (`telnet.protocol`): an RFC 854 NVT codec implemented as a state machine, and an RFC 1143 option negotiator for ECHO, SUPPRESS-GO-AHEAD, STATUS, TIMING-MARK, TERMINAL-TYPE, NAWS and TERMINAL-SPEED.
2. **Server** (`telnet.server`): the five VTY lines of router `JU-Main-R` (`line vty 0 4`), character-at-a-time mode with server-side echo, and a Cisco IOS-style command shell with user, privileged and configuration modes, abbreviations, `?` help and Tab completion.
3. **Client** (`telnet.client`): a console Telnet client that negotiates the same options.
4. **Wire tap** (`telnet.tap`): a man-in-the-middle relay that shows exactly what a sniffer on the path sees, including the reconstructed password.
5. **Telnet over TLS** ("telnets") on a second port, and a **brute-force guard** modelled on IOS `login block-for`.
6. **Live dashboard** on `http://localhost:8081` that follows the lecture slides chapter by chapter.
7. **A high-availability cluster**: three routers behind a TCP load balancer with heartbeats, and **session resume** after a node fails, using session state replicated between the nodes.
8. **AAA with RADIUS** (RFC 2865 / 2866): our own RADIUS server and the router's RADIUS client, for central logins, privilege levels and accounting of every command.

9. **LINEMODE** (RFC 1184) and an experiment that measures character mode, Nagle's algorithm (RFC 896) and LINEMODE on the same typing.

Table 1: Ports used by the Telnet module

Port	Protocol	Role
TCP 2323	Telnet	The server (real port 23 needs administrator rights on Windows).
TCP 2324	Telnet	Wire tap relaying to 2323: shows what a sniffer sees.
TCP 9923	Telnet over TLS	"telnets" (real port 992) with a self-signed certificate made by `keytool`.
TCP 9924	TLS	Wire tap relaying to 9923: shows that TLS hides everything.
HTTP 8081	HTTP	The dashboard.
TCP 2330	Telnet	HA cluster: load balancer in front of routers R1–R3 (TCP 2331–2333).
UDP 2340, 2341–2343	cluster	Heartbeats to the balancer; session-state replication between nodes.
UDP 1812 / 1813	RADIUS	Authentication / accounting (shared secret `testing123`).

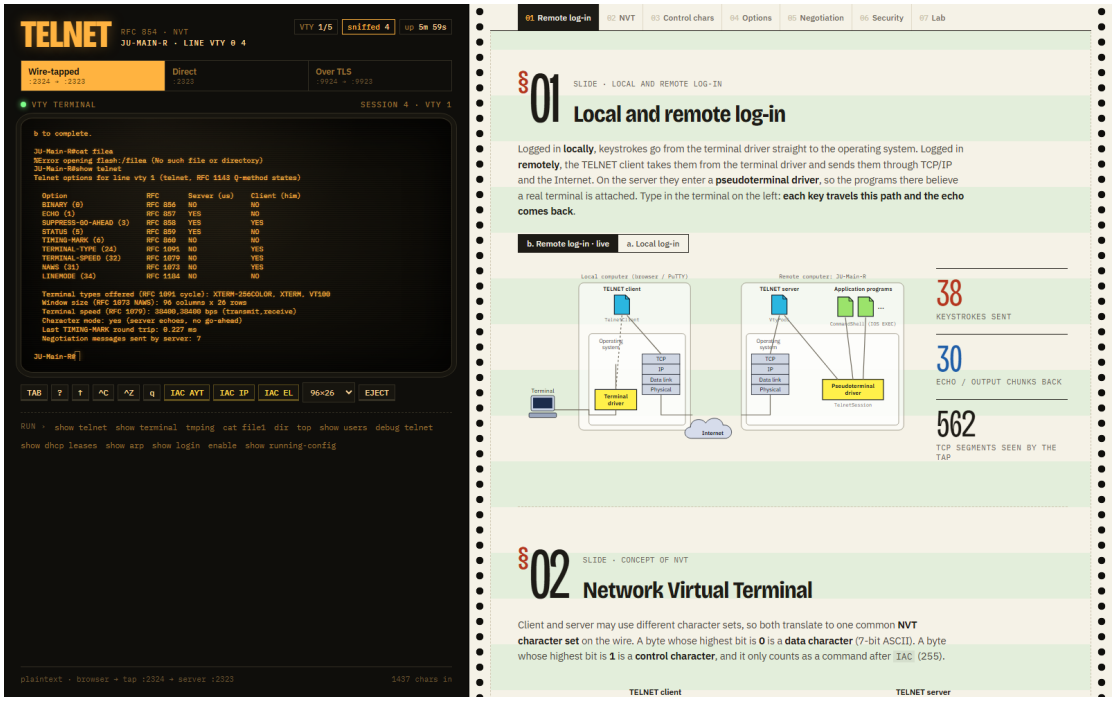


Figure 1: Chapter 01 of the dashboard: the remote log-in path from the lecture slides, with a live browser terminal.

2 The Protocol

2.1 Local and Remote Log-in

On a local log-in, keystrokes go from the terminal to the *terminal driver* of the operating system, which hands them to the shell. On a remote log-in, the Telnet *client* takes them from the terminal driver, converts them to NVT form and sends them over TCP. On the server, the

Telnet *server* cannot hand bytes to the shell directly, because the shell expects a terminal. A **pseudo-terminal driver** sits in between and makes the shell believe a real terminal is attached.

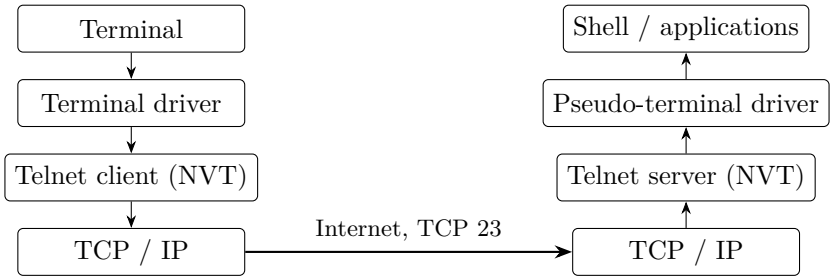


Figure 2: Remote log-in: the path of every keystroke and echo.

2.2 The Network Virtual Terminal and Its Commands

NVT data characters are 7-bit ASCII with the most significant bit 0. End of line is always `CR LF`; a bare carriage return is sent as `CR NUL`. Control is signalled in-band: the byte **IAC (255)** means "the next byte is a command". A data byte that happens to be 255 is therefore sent twice (`FF FF`).

Table 2: Telnet commands (RFC 854) implemented by the codec

Name	Code	Meaning
SE / SB	240 / 250	End / begin of sub-negotiation (`IAC SB option ... IAC SE`).
NOP, DM, BRK	241, 242, 243	No operation, data mark (for urgent data), break.
IP, AO, AYT	244, 245, 246	Interrupt process, abort output, *Are You There?*
EC, EL, GA	247, 248, 249	Erase character, erase line, go ahead.
WILL, WONT	251, 252	"I will (will not) enable this option on my side."
DO, DONT	253, 254	"Please (do not) enable this option on your side."
IAC	255	Interpret As Command; doubled when it is a data byte.

A command sequence on the wire therefore looks like `[IAC][DO][ECHO]` = `FF FD 01`. Sub-negotiation carries values, for example the terminal type:

```
[IAC][SB][24][IS]'x"t"e"r"m'[IAC][SE]
```

2.3 Options

Table 3: Options negotiated by the server and client

Option	RFC	Code	Use in this lab
ECHO	857	1	The server echoes; the password is simply not echoed.
SUPPRESS-GO-AHEAD	858	3	Together with ECHO this gives character-at-a-time mode.
STATUS	859	5	`IAC SB STATUS SEND` returns the options the server believes are on.
TIMING-MARK	860	6	Used by `tmping` to measure round-trip time.
TERMINAL-TYPE	1091	24	The client's type list is cycled until it repeats.
NAWS	1073	31	Window size; the full-screen `top` command reflows on resize.
TERMINAL-SPEED	1079	32	Reported by the client.
BINARY, LINEMODE, ...			Refused correctly with `WONT` / `DONT`.

3 System Architecture

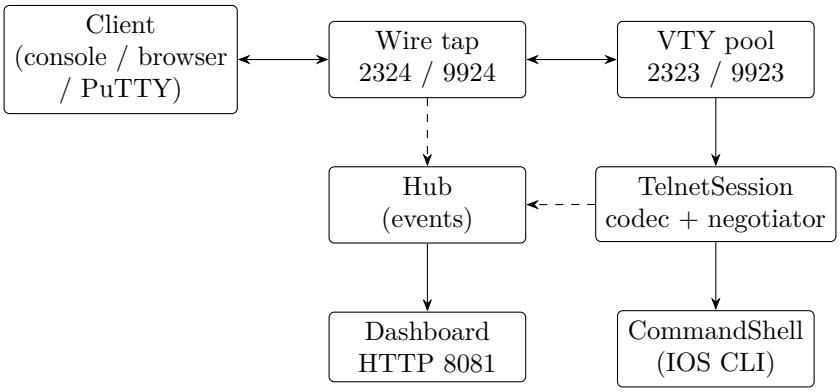


Figure 3: Components of the Telnet module.

Table 4: Source files

Package / file	Responsibility
`protocol/Telnet.java`	Command and option constants with names.
`protocol/TelnetCodec.java`	RFC 854 decoder state machine and encoder.
`protocol/OptionNegotiator.java`	RFC 1143 Q-method, every message labelled with its textbook meaning.
`protocol/LoopExperiment.java`	Naive vs Q-method negotiation experiment.
`server/VtyPool.java`	Five VTY lines; a sixth connection is refused.
`server/TelnetSession.java`	One VTY line: login, line editor, history, options.
`server/CommandShell.java`	IOS-style command tree and modes.
`server/DeviceConfig.java`	Users, PBKDF2 secrets, `login block-for`, `write memory`.
`server/LoginGuard.java`	Brute-force detection and quiet mode.
`tap/WireTap.java`	Sniffing relay and credential reconstruction.
`client/TelnetClient.java`	Console client.
`dashboard/DashboardServer.java`	HTTP API, browser-terminal bridge, attack bot.

4 Implementation

4.1 The NVT Codec as a State Machine

TCP is a byte stream with no message boundaries, so a command can be split across two reads: `FF` at the end of one segment and `FD 1F` at the start of the next. A decoder that looks at whole buffers fails on such input. The codec therefore keeps its state between calls and consumes one byte at a time.

```
private enum State { DATA, IAC, NEGOTIATE, SB_OPTION, SB_DATA, SB_IAC }

public void feed(int b) {
    switch (state) {
        case DATA:
            if (b == Telnet.IAC) state = State.IAC;
            else listener.onData(b);
            break;
        case IAC:
            if (b == Telnet.IAC) { listener.onData(Telnet.IAC); state = State.DATA; }   // escaped 255
            else if (b >= Telnet.WILL && b <= Telnet.DONT) { verb = b; state = State.NEGOTIATE; }
            else if (b == Telnet.SB) { state = State.SB_OPTION; }
            else { listener.onCommand(b); state = State.DATA; }                    // AYT, IP, EC ...
            break;
        case NEGOTIATE:
            listener.onNegotiation(verb, b);
            state = State.DATA;
            break;
        case SB_OPTION: sbOption = b; sb.reset(); state = State.SB_DATA; break;
        case SB_DATA:
            if (b == Telnet.IAC) state = State.SB_IAC; else sb.write(b);
            break;
        case SB_IAC:
            if (b == Telnet.SE) { listener.onSubnegotiation(sbOption, sb.toByteArray()); state = State.DATA; }
            else if (b == Telnet.IAC) { sb.write(Telnet.IAC); state = State.SB_DATA; }
            else { state = State.DATA; }   // malformed: drop the sub-negotiation
            break;
    }
}

/** Escape user data for the wire: 0xFF -> FF FF, bare CR -> CR NUL. */
public static byte[] encodeData(byte[] data) {
    ByteArrayOutputStream out = new ByteArrayOutputStream(data.length + 8);
    for (int i = 0; i < data.length; i++) {
        int b = data[i] & 0xFF;
        out.write(b);
        if (b == Telnet.IAC) out.write(Telnet.IAC);
        if (b == '\r' && (i + 1 >= data.length || data[i + 1] != '\n')) out.write(0);
    }
    return out.toByteArray();
}
```

Listing 1: The RFC 854 decoder in `TelnetCodec.java`.

The verification suite feeds the same byte stream through the decoder in **2000 random segmentations** and checks that every one decodes identically.

Embedding a command in data. The textbook example sends `c a t _ f i l e a IAC EC 1` as one TCP segment. The server's line editor applies the Erase Character command in

place, so the command that runs is `cat file1`. The dashboard has a button that sends exactly this segment.

4.2 Option Negotiation: Why a Naive Implementation Loops

The simple rule "answer every `DO` with `WILL`" breaks when both sides change their mind at the same time: each acknowledges the other's request with a new request, and the messages bounce forever. RFC 1143 gives each side, for each option, one of four states (`NO`, `YES`, `WANTNO`, `WANTYES`) plus a one-bit queue. A side never replies to a message that only confirms its current state, so the exchange always terminates.

```
private void recvWill(int o) {
    Q b = him[o];
    switch (b) {
        case NO:
            if (policy.acceptRemote(o)) { him[o] = Q.YES; send(Telnet.DO, o,
    APPROVE_OFFER_EN); changed(false, o, true); }
            else { send(Telnet.DONT, o, DISAPPROVE_OFFER_EN); }        // refuse the offer
            break;
        case YES:   break;                                            // already on: no reply,
     so no loop
        case WANTNO:
            if (!himQueue[o]) him[o] = Q.NO;                           // error: DONT answered
     by WILL
            else { him[o] = Q.YES; himQueue[o] = false; }
            break;
        case WANTYES:
            if (!himQueue[o]) { him[o] = Q.YES; changed(false, o, true); }   // our DO
    acknowledged
            else { him[o] = Q.WANTNO; himQueue[o] = false; send(Telnet.DONT, o, REQ_DIS); }
            break;
    }
}
```

Listing 2: Receiving WILL under the Q method (`OptionNegotiator.java`).

Every message is also labelled with the meaning it has in the textbook table. The same byte `IAC WILL ECHO` is "*Offering to enable*" when the sender was in state `NO`, but "*Accepting a request to enable*" when it answers our `DO`. The dashboard shows these labels on the live session.

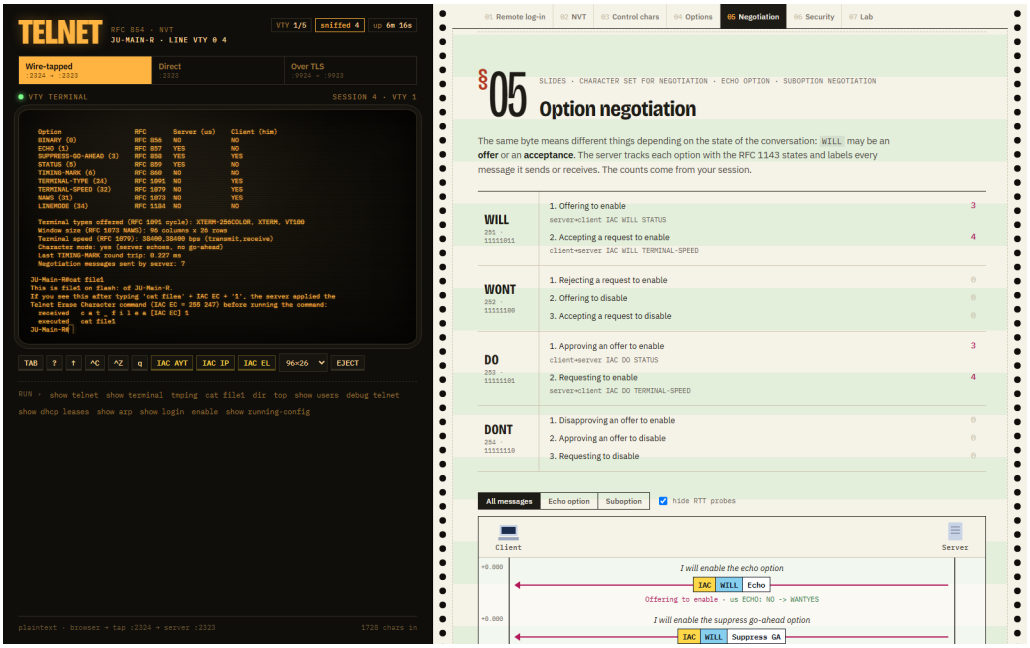


Figure 4: Chapter 05: every WILL/WONT/DO/DONT of the opening negotiation, labelled with its textbook meaning.

The loop experiment. `LoopExperiment.java` connects two endpoints and makes both change their ECHO setting at the same moment. With naive endpoints the messages never stop (the run is cut at a cap); with RFC 1143 endpoints the exchange finishes in **4 messages** and both ends agree on the final state.

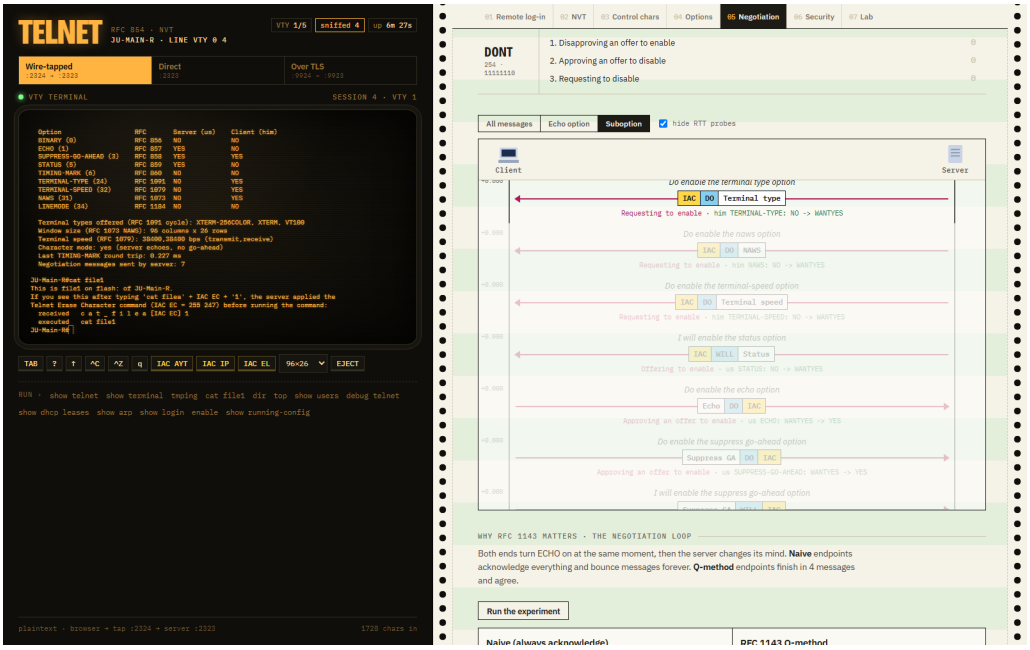


Figure 5: The naive endpoints loop until the cap; the Q-method converges in four messages.

4.3 The Server: VTY Lines and an IOS-Style Shell

`VtyPool` accepts at most five concurrent sessions, like `line vty 0 4`; the sixth connection receives a refusal message and is closed. Each `TelnetSession` starts by offering `WILL ECHO` and

`WILL SUPPRESS-GO-AHEAD` and asking `DO TERMINAL-TYPE`, `DO NAWS` and `DO TERMINAL-SPEED`. With server echo and no go-ahead, every keystroke is sent as soon as it is typed and the server decides what to echo. At the `Password:` prompt it echoes nothing.

The shell implements the IOS command tree with abbreviations (`sh ip int br`), context help with `?`, Tab completion, `% Invalid input` with a `^` marker, user EXEC (`>`), privileged EXEC (`#`) and configuration modes, `enable`, `configure terminal`, `hostname`, `banner motd`, `username ... secret`, `exec-timeout`, `write memory`, `show users`, `show telnet` (the live RFC 1143 state of each option), `tmping` (RTT via TIMING-MARK) and a full-screen `top` that redraws when NAWS reports a new window size. Passwords are stored as **PBKDF2-SHA256** hashes (Cisco type 8) and compared in constant time.

Table 5: Accounts configured on the router

User	Password	Privilege
`admin`	`cisco`	15 (privileged EXEC, `#`)
`student`	`ju2026`	1 (user EXEC, `>`)
enable secret	`class`	raises privilege 1 to 15

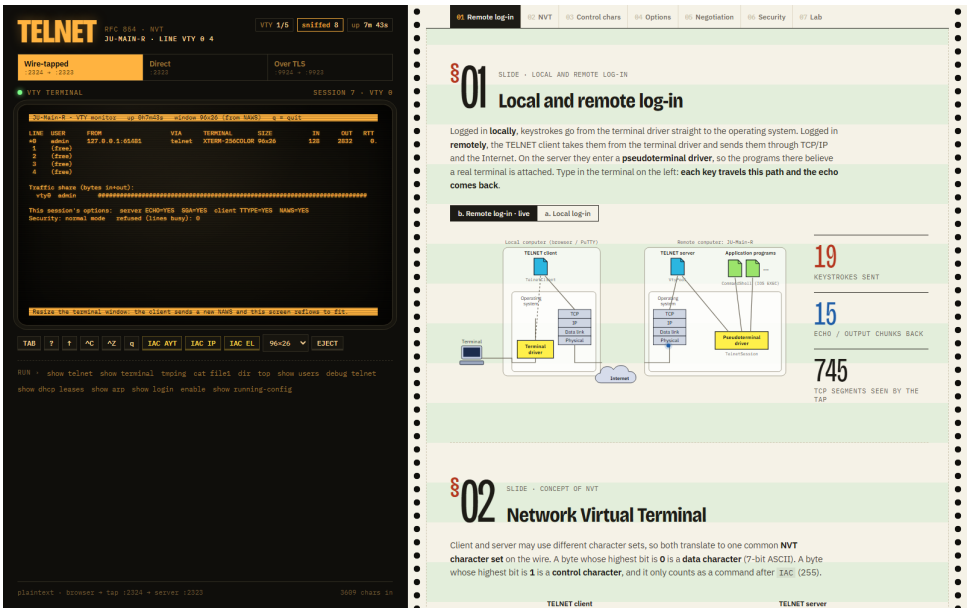


Figure 6: The full-screen `top` command redraws after a NAWS sub-negotiation reports the new window size.

4.4 Security: Sniffing, TLS and Brute-Force Protection

Wire tap. `WireTap` relays every byte unchanged between client and server, decodes the stream for the dashboard, and reconstructs what is typed after the `Username:` and `Password:` prompts. Because Telnet is plaintext, it recovers `admin / cisco` exactly as Wireshark's *Follow TCP Stream* does.

Telnet over TLS. The second listener wraps the same server in TLS with a self-signed certificate generated by `keytool`. Through the TLS tap only record headers are visible; the tap captures no credentials.

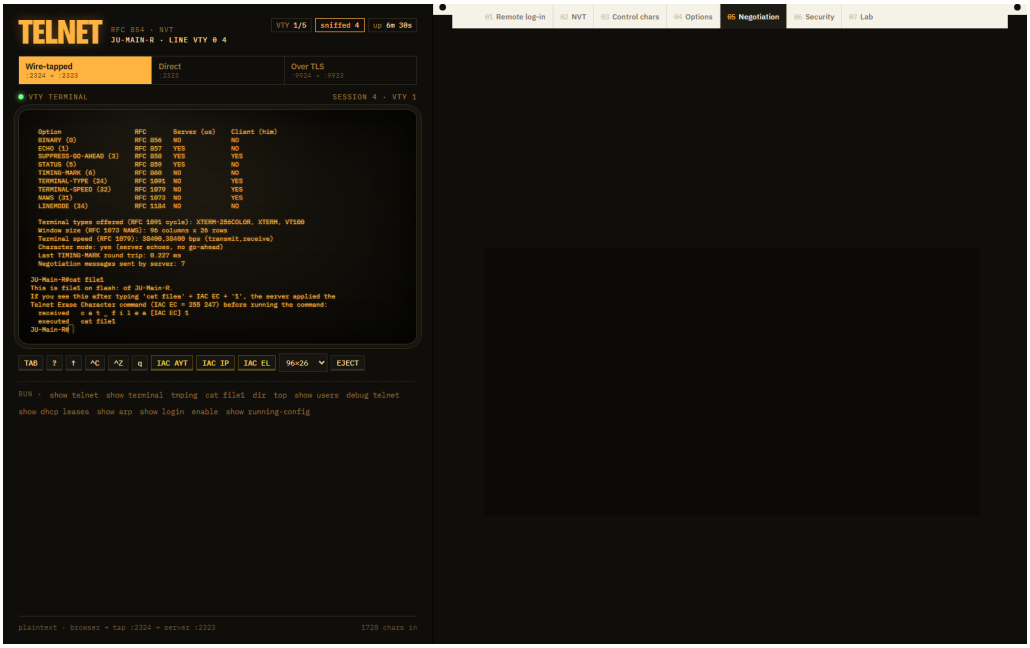


Figure 7: Plaintext Telnet: the password is on the wire. Telnet over TLS: only encrypted records.

Brute-force guard. `LoginGuard` implements `login block-for 30 attempts 3 within 60`: three failures within 60 seconds put the router into *quiet mode* for 30 seconds, during which new connections are refused. The dashboard runs a dictionary-attack bot against it.

```
public synchronized boolean failure(String user, String from) {
    long now = System.currentTimeMillis();
    failures.addLast(now);
    while (!failures.isEmpty() && now - failures.peekFirst() > cfg.blockWithinSec * 1000L)
        failures.removeFirst();                              // forget failures outside the
    window
    if (failures.size() >= cfg.blockAttempts) {
        quietUntil = now + cfg.blockForSec * 1000L;          // %SEC_LOGIN-1-QUIET_MODE_ON
        failures.clear();
        return true;
    }
    return false;
}
```

Listing 3: Sliding-window failure counting in `LoginGuard.java`.

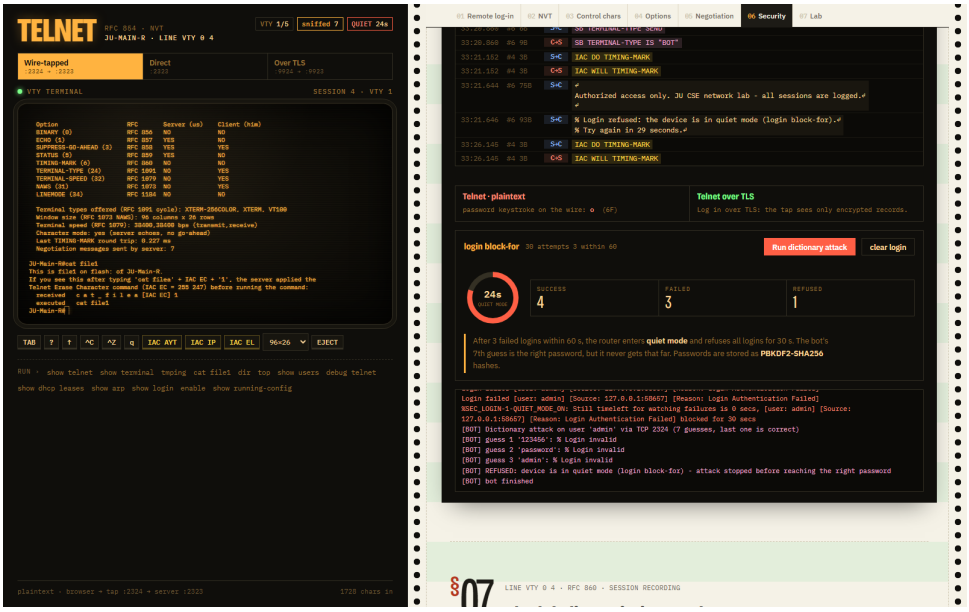


Figure 8: The dictionary attack is stopped after three guesses; the quiet-mode countdown starts.

5 Advanced Features

5.1 High Availability: A Router Cluster with Session Resume

The problem. A router is a single point of failure for management: if it reboots, every Telnet session on it is lost, and so is what the administrator was doing. Load balancing helps new connections, but a Telnet session is *stateful TCP*: the connection's sequence numbers, buffers and the shell behind it live in one machine, and a TCP connection cannot be moved to another host.

The design. Three routers (R1, R2, R3) run the same VTY server behind one **layer-4 TCP load balancer** on port 2330. What *can* move is the session's state, so the cluster replicates it:

- **Health checking.** Each node sends a UDP heartbeat to the balancer every 400 ms. A node silent for 1.5 s is taken out of the rotation, and put back when its heartbeats return.
- **Scheduling.** New connections go to the healthy node with the fewest connections (least connections), in turn (round robin), or by a rendezvous hash of the client address, so one client always lands on the same node while the node set is unchanged.
- **State replication.** After login and after every command, the node sends a snapshot of the session (user, privilege, mode, command history, last screen) as one UDP datagram to its peers; the newest timestamp wins. A logout deletes the snapshot everywhere. A restarted node asks a peer for the whole table (anti-entropy).
- **Resume.** When the connection is lost, the user reconnects to port 2330, lands on a healthy node, and types `resume` (or `resume TOKEN`) at the `Username:` prompt. That node already holds the snapshot and continues the session in the same mode with the same history. The token printed at login works like a session cookie; resuming by client address alone is a lab convenience.

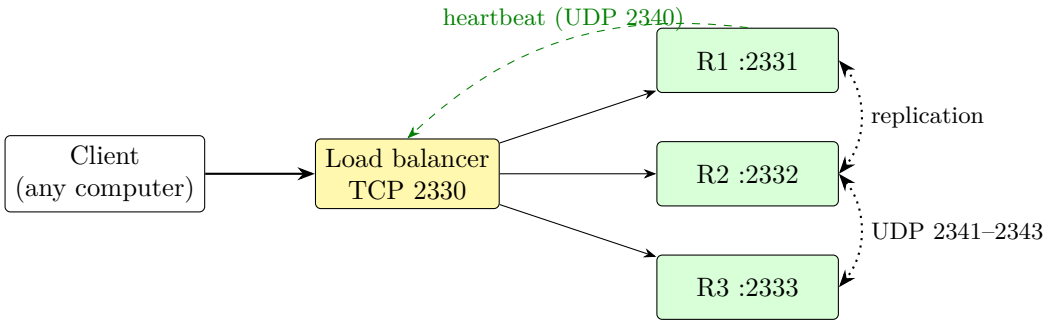


Figure 9: The HA cluster: one public port, three routers, heartbeats and state replication.

```
Node pick(String clientIp) {
    List<Node> up = new ArrayList<>();
    for (Node n : cluster.nodes) if (isHealthy(n)) up.add(n);    // heartbeat seen in the last 1.5 s
    if (up.isEmpty()) return null;
    switch (policy) {
        case "rr":   return up.get(Math.floorMod(rr.getAndIncrement(), up.size()));
        case "hash": {                                           // rendezvous hashing: stable when nodes come and go
            Node best = null; long bestScore = Long.MIN_VALUE;
            for (Node n : up) { long sc = mix((clientIp + "|" + n.name).hashCode()); if (sc > bestScore) { bestScore = sc; best = n; } }
            return best;
        }
        default: {                                               // least connections
            Node best = null; int bestN = Integer.MAX_VALUE;
            for (Node n : up) { int k = count(n); if (k < bestN) { bestN = k; best = n; } }
            return best;
        }
    }
}

/** Store locally and send to every other node. A crashed node sends nothing: it is dead. */
void replicate(Snapshot x) {
    if (!alive()) return;
    store.put(x.token, x);
    for (Node n : cluster.nodes) if (n != this) sendTo(n.replPort, x.encode());
}
```

Listing 4: Choosing a node and replicating a snapshot (`cluster/Balancer.java`, `cluster/Node.java`).

A box in the middle reads plaintext. The balancer does not terminate Telnet; it copies bytes between the client and the chosen node. Because Telnet is plaintext, it can still read everything, exactly like the wire tap: it runs the same NVT decoder on both directions and recovers the user name and password typed at the login prompts. The dashboard therefore shows cluster sessions the same way as tapped ones, and the whole demonstration (sniffing, failover and RADIUS logins) runs from another computer through one port, 2330. In a real network this is a reason never to send management traffic in clear text through load balancers or proxies.

Measured behaviour. In the verification run an administrator logged in through port 2330 and landed on R1, ran `show clock` and entered configuration mode; the snapshot was present on all three nodes. R1 was then killed: the client's TCP connection dropped at once, and 1.5 s later the balancer had removed R1. The reconnection landed on R2, and `resume` continued the session *in configuration mode*, with `show clock` and `configure terminal` in its history. After

logout no copy of the snapshot remained, and the revived R1 rejoined the rotation.

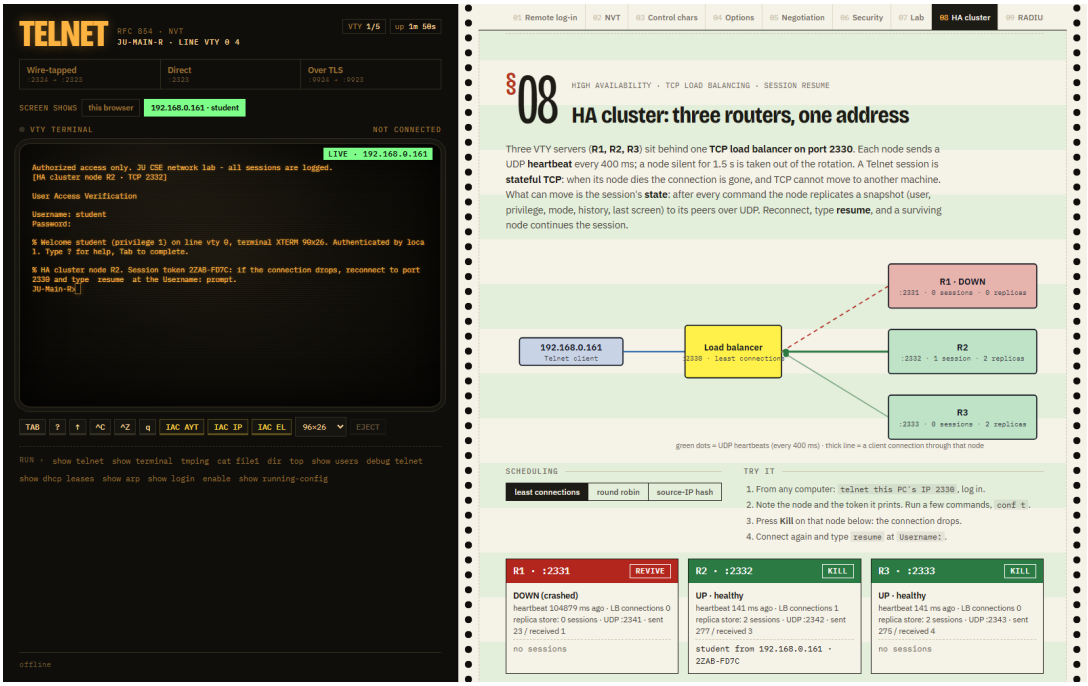


Figure 10: Chapter 08 of the dashboard after R1 was killed: R1 is out of the rotation, and the sessions continue on R2 and R3.

5.2 AAA: Central Login and Accounting with RADIUS

Storing passwords on every router does not scale: adding or removing an administrator means changing every device, and nobody can see who did what. Real networks use **AAA** (authentication, authorisation, accounting) against a central server. RADIUS (RFC 2865, UDP 1812) and RADIUS accounting (RFC 2866, UDP 1813) were implemented on both sides: a server with its own user table, and the router's client (the *NAS*, network access server).

Table 6: RADIUS packet layout and the packets used

Field	Content
Header (20 bytes)	Code (1), Identifier (1), Length (2), Authenticator (16)
Attributes	Type (1), Length (1), Value: User-Name, User-Password, NAS-IP-Address, NAS-Port, NAS-Port-Type (Virtual), Calling-Station-Id (client address), NAS-Identifier (router name), Service-Type, Reply-Message, Vendor-Specific (Cisco AV-pair), Acct-Status-Type, Acct-Session-Id, Acct-Session-Time, Acct-Input/Output-Octets, Acct-Terminate-Cause
Codes	1 Access-Request, 2 Access-Accept, 3 Access-Reject, 4 Accounting-Request, 5 Accounting-Response

Security from a shared secret. RADIUS does not encrypt the packet; it protects it with a secret S known to the router and the server:

- The password is hidden: $c_1 = p_1 \oplus \mathrm{MD5}(S \,\|\, \mathit{RequestAuthenticator})$, $c_i = p_i \oplus \mathrm{MD5}(S \,\|\, c_{i-1})$, in 16-byte blocks.
- Every reply carries $\mathrm{MD5}(\mathit{Code} \,\|\, \mathit{ID} \,\|\, \mathit{Length} \,\|\, \mathit{RequestAuthenticator} \,\|\, \mathit{Attributes} \,\|\, S)$, so the router can tell a genuine reply from a forged one.

- An Accounting-Request carries the same MD5 over the packet with 16 zero bytes in place of the authenticator, so the server can reject forged accounting records.

```
/** RFC 2865 section 5.2 User-Password hiding. */
public static byte[] hidePassword(String password, byte[] secret, byte[] requestAuth) {
    byte[] p = password.getBytes(StandardCharsets.UTF_8);
    int n = Math.max(16, (p.length + 15) / 16 * 16);
    byte[] padded = new byte[n];
    System.arraycopy(p, 0, padded, 0, p.length);
    byte[] out = new byte[n], prev = requestAuth;
    for (int i = 0; i < n; i += 16) {
        byte[] b = md5(secret, prev);
        for (int j = 0; j < 16; j++) out[i + j] = (byte) (padded[i + j] ^ b[j]);
        prev = Arrays.copyOfRange(out, i, i + 16);
    }
    return out;
}

// on the router: accept a reply only if its authenticator proves the server knows the
    secret
if (!MessageDigest.isEqual(r.auth, Radius.responseAuth(r, q.auth, key))) {
    badResponses++;            // wrong shared secret or a forged reply: discard it
    continue;
}
```

Listing 5: Hiding the password and checking a reply (`aaa/Radius.java`, `aaa/RadiusClient.java`).

Login flow. With `aaa authentication login default group radius local` configured, the router sends an Access-Request for every login and retransmits the identical packet after 1 s, up to three times. An Access-Accept carries the privilege level as the Cisco AV-pair `shell:priv-lvl=15`; an Access-Reject is final. Only if the server never answers does the router fall back to its local users. For each exec session the router sends an Accounting Start, one Interim-Update per command (with the command in a Cisco AV-pair), and a Stop with the session time, the byte counts and the terminate cause. Command accounting is normally done by Cisco with TACACS+; carrying it in RADIUS Interim-Updates is our extension to show the idea on the protocol we implemented.

Table 7: RADIUS behaviour observed in the verification run

Situation	**Result**
User `netops` (exists only on the RADIUS server)	Access-Accept; logged in with privilege 15 from the AV-pair
`admin` with a wrong password	Access-Reject (“Wrong password”); no fallback to the local password
Server offline	Three retransmissions, then local login for `admin`; `netops` cannot log in
Router configured with the wrong shared secret	Replies fail the Response Authenticator check and are discarded
Accounting	Start, Interim-Update with `show users`, and Stop recorded on the server

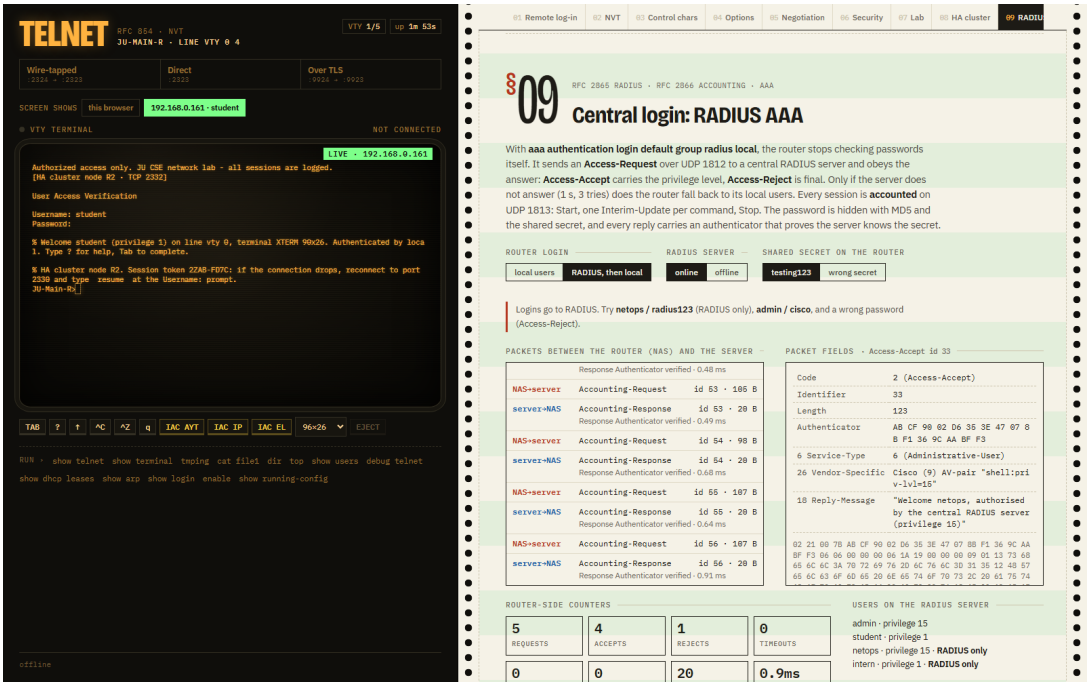


Figure 11: Chapter 09: the RADIUS packets between the router and the server, decoded field by field.

5.3 LINEMODE, Nagle's Algorithm and Tinygrams

In character mode every keystroke is sent as its own TCP segment and echoed in another: one byte of data inside 40 bytes of TCP/IP header, a *tinygram*. Two classic remedies were measured:

- **Nagle's algorithm** (RFC 896): while sent data is unacknowledged, hold new small data and send it together when the acknowledgement arrives. In character mode the echo plays the role of the acknowledgement.
- **LINEMODE** (RFC 1184): the client offers `WILL LINEMODE`; the server answers `DO LINEMODE` and `SB LINEMODE MODE EDIT`, and gives up remote echo. The client then edits and echoes the line itself and sends it complete.

The server was extended to accept LINEMODE. A scripted client typed the same three commands (`show clock`, `who`, `show history`, 28 keystrokes) into the real server three times, through a link that delays every chunk by RTT/2 and counts every segment.

Table 8: Measured at RTT 200 ms and 60 ms per keystroke

	Character mode	**+ Nagle's rule**	**LINEMODE**
Segments client → server	29	12	4
Segments server → client	38	32	10
Bytes on the wire (with 40-byte headers)	3074	2154	923
Header share of the traffic	87%	82%	61%
Average echo delay per keystroke	218 ms	309 ms	0 ms (local)
Command answer time (Enter → prompt)	217 ms	268 ms	218 ms

Character mode sent one segment per key and its echo took a full round trip. Nagle's rule cut the client's segments by more than half but made each echo wait for the previous one.

LINEMODE sent one segment per command and cut the traffic to under a third, with no echo delay at all; the price is that the server no longer sees each key, so character-level features such as `?` help and Tab completion stop working while it is on.

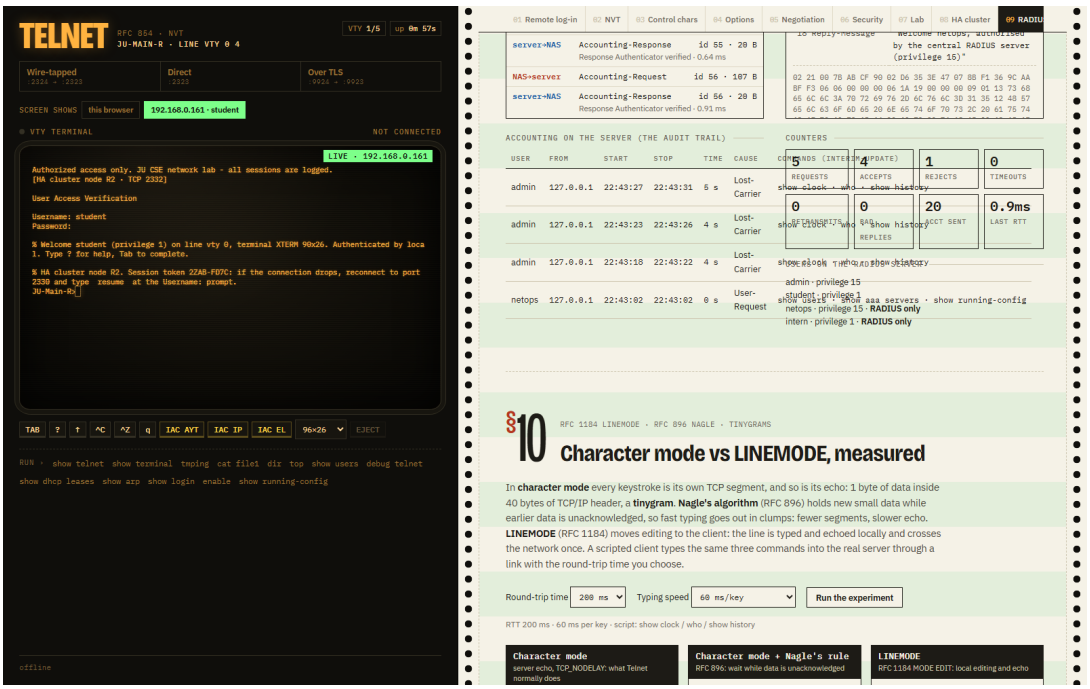


Figure 12: Chapter 10: the three modes measured side by side, with a timeline of every segment.

6 Experiments, Verification and Results

6.1 Running the Module

```
cd Telnet
run.bat                  # compile and start server, taps and dashboard (http://localhost
    :8081)
run.bat --verify         # 59 automated end-to-end checks
run.bat client           # console client through the wire tap
telnet localhost 2323    # any Telnet client works (Windows: enable "Telnet Client")
telnet <this-PC> 2330    # from a second computer: the whole demo through one port
                         # (load balancer: sniffed, failover + resume, RADIUS logins)
```

Listing 6: Starting the lab, running the checks, and connecting by hand.

6.2 Automated Verification

`Verify.java` starts the real server, taps, cluster and RADIUS server, and drives them over TCP and UDP. All **59 checks pass**: 35 for the protocol, server and security, 10 for the cluster, 9 for RADIUS and 5 for the LINEMODE experiment. A selection:

Table 9: Selected verification checks (all passed)

0xFF in data is doubled, bare CR becomes CR NUL
Fuzz: 2000 random TCP segmentations decode identically
Naive negotiation loops forever (stopped at the cap); the Q-method converges and both ends agree
`DO ECHO` answered once with `WILL`; a repeated `DO` gets no reply
Unsupported option (LINEMODE) refused with `WONT`; unwanted offer (NEW-ENVIRON) refused with `DONT`
TERMINAL-TYPE cycled until it repeats (RFC 1091); NAWS window size received (RFC 1073)
The password was not echoed back by the server
Privilege 1 cannot see `running-config`; `enable` with the enable secret gives `#`
`tmping`: three RTT measurements via TIMING-MARK (RFC 860)
`IAC AYT` answered with a visible `[host:  yes]`
Textbook embedding: `cat filea IAC EC 1` runs `cat file1`
STATUS (RFC 859): `SB STATUS SEND` answered with `SB STATUS IS`
Wire tap sniffs `admin / cisco` from plaintext Telnet; through TLS it captures no credentials
Sixth connection refused: all five VTY lines busy
Three failed logins trigger quiet mode; the next connection is refused; `clear login` ends it
Cluster: snapshot on all three nodes; killing the node drops the client; the balancer removes it after 1.5 s; `resume` on another node restores configuration mode and history
RADIUS: password hiding reverses only with the right secret; RADIUS-only login; Reject is final; fallback when the server is silent; wrong secret detected; accounting recorded
LINEMODE experiment: one segment per key in character mode; fewer segments but slower echo with Nagle; one segment per line in LINEMODE

6.3 Observations

1. **Character mode costs one segment per keystroke.** With ECHO and SUPPRESS-GO-AHEAD on, each key is a separate TCP segment and its echo another, which is why the sniffer sees the password one character at a time.

2. **RFC 1143 terminates where the naive rule does not.** Under simultaneous changes the naive endpoints exchanged messages until the experiment's cap; the Q-method needed four.

3. **Plaintext is the real problem.** The same login through the plaintext tap reveals the credentials; through TLS it reveals nothing but record lengths.

4. **One port, real network.** From a second laptop on the same Wi-Fi, `telnet <IP> 2330` reached the cluster; the dashboard showed that laptop's address, its negotiation and its sniffed password, a RADIUS-only user logged in through the same port with privilege 15, and the accounting record carried the laptop's address as Calling-Station-Id.

5. **Interoperation.** Windows `telnet.exe` and PuTTY log in to the server and negotiate the same options as our client. In Wireshark (loopback adapter, `tcp.port == 2324`, *Decode As TELNET*) the negotiation and the password appear exactly as on the dashboard.

6.4 Use in the Integrated Main Project

In the integrated main project the same server is the management interface of router `JU-Main-R` on the campus LAN. A `CampusHooks` interface lets the CLI show the DHCP-snooping bindings and ARP tables of the other modules and switch Dynamic ARP Inspection on with `ip arp inspection vlan 1`. The Telnet bytes are carried as frames across the ARP module's switch, so ARP poisoning really redirects them to the attacker.

7 Conclusion

Building Telnet from the RFCs made its design concrete:

1. **In-band signalling needs a stateful decoder.** Because commands share the byte stream with data and TCP has no message boundaries, the NVT decoder must be a state machine that survives any segmentation, and data byte 255 must be escaped.
2. **Symmetric negotiation needs RFC 1143.** Labelling each WILL/WONT/DO/DONT with its meaning and tracking four states plus a queue per option prevents negotiation loops that the naive rule cannot avoid.
3. **State can survive what TCP cannot.** A TCP connection dies with its server, but replicating the session's state lets another node continue it; the cluster resumed a session in configuration mode after its node was killed.
4. **Management belongs on a central AAA server.** With RADIUS, one server decided who could log in to the routers and with what privilege, recorded every command, and the shared-secret authenticators exposed a misconfigured router.
5. **Character-at-a-time costs bandwidth, and every fix costs something else.** Nagle's algorithm halved the segments at the price of echo delay; LINEMODE cut the traffic to under a third at the price of per-key features.
6. **Remote log-in must be encrypted.** The wire tap recovered the administrator password from plaintext Telnet, while Telnet over TLS exposed nothing. Rate-limited logins slow guessing attacks but do not replace encryption, which is the reason SSH superseded Telnet.

AI Acknowledgement

Artificial Intelligence tools were used as an aid during this project. Specifically, AI assistance was used for:

- **Protocol cross-checking:** comparing message formats, reply codes, state machines and byte offsets against the RFCs cited in this report.
- **Code review and testing:** suggesting edge cases for the automated verification suite and reviewing concurrency and error handling.
- **Writing and visualization:** drafting, structuring and refining the explanations, tables, diagrams and LaTeX source of this report.

All protocol implementations, Java source code, experiments, measurements and final conclusions were run, reviewed and verified by the author, who takes full responsibility for the content of this report.